

```

import numpy as np
import matplotlib.pyplot as plt

# =====
# Task 1: Numerical Gradient (Central Difference)
# =====

print("===== Task 1 =====")

def compute_gradient(f, x):
    h = 1e-5
    return (f(x + h) - f(x - h)) / (2 * h)

# Example function
def f(x):
    return x**2

x = 3
gradient = compute_gradient(f, x)

print("Function: f(x) = x^2")
print("Gradient at x =", x, "=", gradient)

# =====
# Task 2: Partial Derivatives
# f(x,y) = x^2 + y^2
# =====

print("\n===== Task 2 =====")

def partial_x(x, y):
    return 2 * x

def partial_y(x, y):
    return 2 * y

x = 3
y = 4

print("∂f/∂x =", partial_x(x, y))
print("∂f/∂y =", partial_y(x, y))

```

```
# =====  
# Task 3: One Step Gradient Descent  
#  $f(w) = (w-5)^2$   
# =====
```

```
print("\n===== Task 3 =====")
```

```
learning_rate = 0.1  
w = 0
```

```
gradient = 2 * (w - 5)
```

```
w = w - learning_rate * gradient
```

```
print("Updated Weight =", w)
```

```
# =====  
# Task 4: Learning Rate Comparison  
# =====
```

```
print("\n===== Task 4 =====")
```

```
def gradient(x):  
    return 2 * (x - 3)
```

```
learning_rates = [0.01, 0.1, 0.5]
```

```
iterations = 20
```

```
plt.figure(figsize=(8,6))
```

```
for lr in learning_rates:
```

```
    x = 0  
    history = []
```

```
    for i in range(iterations):  
        history.append(x)  
        x = x - lr * gradient(x)
```

```
    plt.plot(history, label=f"Learning Rate = {lr}")
```

```

plt.title("Gradient Descent with Different Learning Rates")
plt.xlabel("Iterations")
plt.ylabel("x Value")
plt.legend()
plt.grid(True)
plt.show()

# =====
# Task 5: Plot Function
# =====

print("\n===== Task 5 =====")

def plot_function():

    x = np.linspace(-2,4,400)

    y = x**4 - 3*x**3 + 2

    plt.figure(figsize=(8,6))
    plt.plot(x,y)

    plt.title("f(x)=x4-3x3+2")
    plt.xlabel("x")
    plt.ylabel("f(x)")
    plt.grid(True)

    # Local/Global Minimum (Approx.)
    plt.scatter(2.25, 2.25**4 - 3*(2.25**3) + 2,
               color='red',
               label="Minimum")

    plt.legend()

    plt.show()

plot_function()

```

Explanation

Task 1: Numerical Gradient (Central Difference)

Formula

$$f'(x) = \frac{f(x+h) - f(x-h)}{2h}$$

where:

- $h = 10^{-5}$

Example:

$$f(x) = x^2$$

At

$$x = 3$$

Actual derivative

$$2x = 6$$

The numerical gradient computed by the program will be very close to **6**, showing how derivatives can be approximated without symbolic differentiation.

Task 2: Partial Derivatives

Given

$$f(x, y) = x^2 + y^2$$

Partial derivative with respect to x

$$\frac{\partial f}{\partial x} = 2x$$

Partial derivative with respect to y

$$\frac{\partial f}{\partial y} = 2y$$

At

$$x=3$$

$$y=4$$

Results:

$$\partial f / \partial x = 6$$

$$\partial f / \partial y = 8$$

Task 3: One Step of Gradient Descent

Function

$$f(w) = (w - 5)^2$$

Derivative

$$\frac{df}{dw} = 2(w - 5)$$

Initial value

$$w = 0$$

Gradient

$$2(0-5) = -10$$

Update rule

$$w_{new} = w - \eta \cdot gradient$$

where

$$\eta = 0.1$$

Calculation:

$$w = 0 - 0.1(-10)$$

$$w = 1$$

So after one gradient descent step:

Updated Weight = 1

Task 4: Effect of Different Learning Rates

The script compares three learning rates:

- **0.01:** Small updates, slow convergence.
- **0.1:** Balanced updates, converges efficiently.

- **0.5:** Large updates, reaches the minimum very quickly for this simple quadratic function.

The plot shows how the value of **x** changes over **20 iterations** for each learning rate.

Task 5: Plotting the Function

Function:

$$f(x) = x^4 - 3x^3 + 2$$

The script:

1. Generates x values from **-2 to 4**.
2. Computes the corresponding y values.
3. Plots the curve using Matplotlib.
4. Marks an approximate minimum with a red point.